

Javaプログラミング入門

Introduction to Java Programming

第9回 クラス・文字・文字列

Class 9. Class, Characters and Strings

早稲田大学 基幹理工学部 情報通信学科 講師

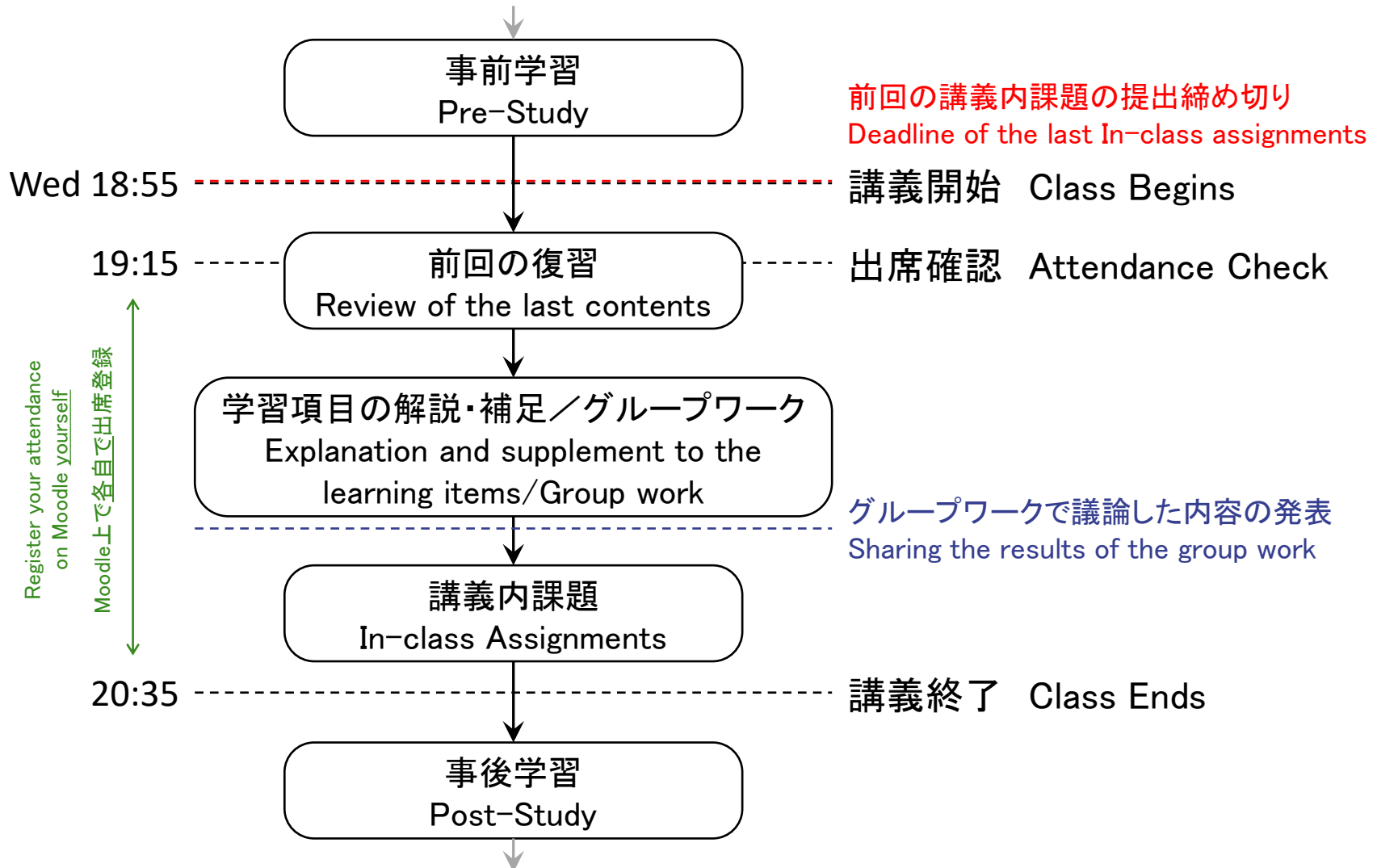
吉井 一駿

Kazutoshi YOSHII

講義の流れ Flow of the Class

本講義は反転講義です。Moodle上にある予習ビデオを視聴の上講義に臨むこと。

This course is a flipped classroom. Watch the video before each class.



第9回講義 Class 9

クラス・文字・文字列 Class, Characters and Strings



第8回 (1) クラス 00:15:14



第8回 (2) 文字・文字列 00:08:24



Part8 (1) Class 00:10:17



Part8 (2) Characters and Strings 00:05:54

到達目標：下記キーワードの理解

Goals for the Next Class: Understanding the Following Topics

クラス Class オブジェクト Object インスタンス
Instance コンストラクタ Constructor フィールド Field



前回の復習

Review of the previous class

可読性の高いプログラム Readable Code

- プログラムが複雑化すると、ソースコードの行数が増えていく。これによりプログラムの全体像を把握することが難しくなり、保守・運用の障害となり得る。

As a program becomes more complex, the number of lines in the source code increases. This makes it difficult to understand the overall structure of the program and may become an obstacle to maintenance and operation.

- コメントやインデントを適切に付与することで、内容や構造的な読みやすさを向上させることができるが、全体のコード量を減らすことはできない。

Adding appropriate comments and indentation can improve the readability of the content and structure.

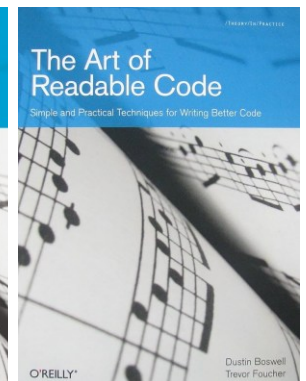
However, they cannot reduce the total amount of code.

- **メソッド**を適切に活用すれば、同じ内容の処理を一度だけ記述すれば目的が果たせるようになり、特に複雑なプログラムにおいてソースコードの総量の削減が期待できる。

By using **methods** appropriately, repeated processes can be written only once. Therefore, especially in complex programs, methods can help reduce the total amount of source code.

参考資料 Recommended Reading

- Dustin Boswell, Trevor Foucher (角 征典 訳), "リーダブルコード", オライリー・ジャパン, 2012. ISBN 978-4-87311-565-8
<https://www.oreilly.co.jp/books/9784873115658/>
- Dustin Boswell, Trevor Foucher, "The Art of Readable Code", O'Reilly Media, Inc., 2011.
<https://www.oreilly.com/library/view/the-art-of/9781449318482/>



メソッドはなぜ必要か Why Methods Are Necessary

- 機能を増やせば増やすほど、プログラムは長くなります。
As more features are added, the program becomes longer.
- プログラムを整理すると、類似した処理が存在することがわかります。
After organizing the program, we can identify similar processes.

メソッドを活用することで、シンプルな記述が可能になります。 By using methods, we can write these processes more simply.



モンスターやプレイヤーが増えても簡単に対応可能。

The program can be easily extended when more monsters or players are added.

MonsterBattleGame_WithMethod.java

```
import java.util.Random; // Randomクラスの使用 / Use the Random class
import java.util.Scanner; // Scannerクラスの使用 / Use the Scanner class

public class MonsterBattleGame_WithMethod {

    public static void main(String args[]){
        // -----
        // プログラム実行の準備 / Program execution setup
        Scanner scan = new Scanner(System.in); // 入力処理の準備 / Prepare input handling
        Random rand = new Random(100); // 乱数生成処理の初期化 / Initialize random number generation

        // -----
        // パラメータ定義 / Parameters definition
        int player_hp = 100; // プレイヤーのHP / Player HP
        int monster_hp = 100; // モンスターのHP / Monster HP

        // -----
        // バトルプロセス / Battle process
        while (true) {

            System.out.println("Press Enter to attack.");
            scan.nextLine(); // Enterの入力待ち / Wait for Enter input

            // -----
            // プレイヤーの攻撃 / Attack by Player
            monster_hp = attack("Player", "Monster", monster_hp, rand);

            // 勝敗判定 / Check win or lose
            if (monster_hp <= 0) {
                System.out.println("You win!");
                break;
            }

            System.out.println("---");

            // -----
            // 相手の攻撃 / Attack by Monster
            player_hp = attack("Monster", "Player", player_hp, rand);

            // 勝敗判定 / Check win or lose
            if (player_hp <= 0) {
                System.out.println("You lose ...");
                break;
            }
        }
    }
}
```

メソッド化された攻撃プロセス
Attack Process Implemented as a Method

```
// -----
// 攻撃処理 / Attack process
public static int attack(String attacker_name, String target_name, int target_hp, Random rand){

    int max_random_val = 100; // クリティカル判定用乱数の上限 / Upper bound for critical hit random value
    int max_attack_point = 30; // 攻撃ダメージの上限 / Upper bound for attack damage

    int attack_multiplier = 1; // 通常の攻撃倍率 / Normal attack multiplier

    int critical_hit_multiplier = 2; // クリティカルヒット倍率 / Critical hit multiplier
    int critical_hit_rate = 30; // クリティカルヒット発生率 / Critical hit chance

    // 攻撃者を表示 / Show attacker
    System.out.println(attacker_name + "の攻撃 / Attack by " + attacker_name);

    // クリティカルヒット判定 / Check critical hit
    if (rand.nextInt(max_random_val) < critical_hit_rate) {
        attack_multiplier = critical_hit_multiplier;
        System.out.println(" クリティカルヒット! / Critical hit!");
    }

    // 攻撃ダメージを計算 / Calculate attack damage
    int attack_damage = attack_multiplier * rand.nextInt(max_attack_point);

    // 攻撃対象のHPを減らす / Reduce target HP
    target_hp -= attack_damage;

    // 攻撃結果を表示 / Show attack result
    System.out.println("> " + target_name + "に" + attack_damage + "のダメージ! / Dealt " + attack_damage + " damage to the " + target_name + "!");
    System.out.println("> " + target_name + "の残りHP: " + target_hp + " / " + target_name + " HP: " + target_hp);

    // 更新後のHPを返す / Return updated HP
    return target_hp;
}

// public class MonsterBattleGame_WithMethod { はここで閉じる / ends here.
```

講義内課題 A In class Assignment A

MonsterBattleGame.javaの作成

Creating the MonsterBattleGame.java

講義中に作成したMonsterBattleGame.javaの名前を変更して,
下で指定されたファイル名で, コンパイル&実行できる状態で提出して下さい.

Rename the MonsterBattleGame.java file created during class and submit it
use the file name specified below, and submit the program that can be compiled and run.

提出ファイル名 Submission file name: Assignment8A_[Student ID].java

提出方法 Submission method: Moodle

提出期限 Submission deadline: 第9回講義の開始まで. By the start of the class 9.

講義内課題 A In class Assignment A

Assignment8A.java

```
import java.util.Random; // Randomクラスの使用 / Use the Random class
import java.util.Scanner; // Scannerクラスの使用 / Use the Scanner class

public class Assignment8A {
    public static void main(String args[]){

        // -----
        // プログラム実行の準備 / Program execution setup
        Scanner scan = new Scanner(System.in); // 入力処理の準備 / Prepare input handling
        Random rand = new Random(100); // 乱数生成処理の準備 / Prepare random number generation

        // -----
        // パラメータ定義 / Parameters definition
        int player_hp = 100; // プレイヤーのHP / Player HP
        int monster_hp = 100; // モンスターのHP / Monster HP

        int max_random_val = 100; // クリティカル判定用乱数の上限 / Upper bound for critical hit random value

        int max_attack_point = 30; // 攻撃ダメージの上限 / Upper bound for attack damage
        int attack_damage; // 実際に与えるダメージ / Actual attack damage
        int player_attack; // プレイヤーの攻撃値 / Player's attack value
        int monster_attack; // モンスターの攻撃値 / Monster's attack value

        int critical_hit_rate = 30; // クリティカルヒット発生率 / Critical hit chance
        int critical_hit_multiplier = 2; // クリティカルヒット倍率 / Critical hit multiplier
        int attack_multiplier = 1; // 通常時の攻撃倍率 / Normal attack multiplier

        // -----
        // バトルプロセス / Battle process
        while (true) {

            System.out.println("Press Enter to attack.");
            scan.nextLine(); // Enterの入力待ち / Wait for Enter input

            // -----
            // プレイヤーの攻撃 / Attack by Player
            System.out.println("プレイヤーの攻撃 / Attack by Player");

            // クリティカルヒット判定 / Check critical hit
            if (rand.nextInt(max_random_val) < critical_hit_rate) {
                attack_multiplier = critical_hit_multiplier;
                System.out.println("> クリティカルヒット! / Critical hit!");
            }

            // プレイヤーの攻撃ダメージを計算 / Calculate player's attack damage
            player_attack = attack_multiplier * rand.nextInt(max_attack_point);

            // 実際のダメージとして設定 / Set actual attack damage
            attack_damage = player_attack;

            // モンスターのHPを減らす / Reduce monster HP
            monster_hp -= attack_damage;

            // クリティカル倍率を通常値に戻す / Reset critical hit multiplier
            attack_multiplier = 1;

            // 攻撃結果を表示 / Show attack result
            System.out.println("> モンスターに" + attack_damage + "のダメージ! / Dealt " + attack_damage + " damage to the monster!");
            System.out.println("> モンスターの残りHP: " + monster_hp + " / Monster HP: " + monster_hp);

            // 勝利判定 / Check win or lose
            if (monster_hp <= 0) {
                System.out.println("You win!");
                break;
            }

            // -----
            // 相手の攻撃 / Attack by Monster
            System.out.println("モンスターの攻撃 / Attack by Monster");

            // クリティカルヒット判定 / Check critical hit
            if (rand.nextInt(max_random_val) < critical_hit_rate) {
                attack_multiplier = critical_hit_multiplier;
                System.out.println("> クリティカルヒット! / Critical hit!");
            }

            // モンスターの攻撃ダメージを計算 / Calculate monster's attack damage
            monster_attack = attack_multiplier * rand.nextInt(max_attack_point);

            // 実際のダメージとして設定 / Set actual attack damage
            attack_damage = monster_attack;

            // プレイヤーのHPを減らす / Reduce player HP
            player_hp -= attack_damage;

            // クリティカル倍率を通常値に戻す / Reset critical hit multiplier
            attack_multiplier = 1;

            // 攻撃結果を表示 / Show attack result
            System.out.println("> プレイヤーに" + attack_damage + "のダメージ! / Player took " + attack_damage + " damage!");
            System.out.println("> プレイヤーの残りHP: " + player_hp + " / Player HP: " + player_hp);

            // 勝利判定 / Check win or lose
            if (player_hp <= 0) {
                System.out.println("You lose ...");
                break;
            }
        }
    }
}
```


講義内課題 B In class Assignment B

2体の敵モンスターと戦うゲーム, `MonsterBattleGame_2.java` を作成しなさい.

ただし, 以下の条件を満たすこと.

- プレイヤーのHPは200, モンスターのHPはそれぞれ100.
- 20%の確率で攻撃が失敗するようにプログラムすること.
- 乱数のシードは999とすること.

提出の際は, 下で指定されたファイル名で, コンパイル&実行できる状態で提出して下さい.

Create `MonsterBattleGame_2.java`, a game in which the player fights against two enemy monsters.

However, the following conditions must be satisfied.

- The player's HP must be 200, and each monster's HP must be 100.
- Program the attack so that it fails with a probability of 20%.
- Set the random seed to 999.

When submitting, use the file name specified below, and submit the program that can be compiled and run.

提出ファイル名 Submission file name: `Assignment8B_[Student ID].java`

提出方法 Submission method: `Moodle`

提出期限 Submission deadline: `第9回講義の開始まで. By the start of the class 9.`

講義内課題 B In class Assignment B

MonsterBattleGame_WithMethod.java

```
import java.util.Random; // Randomクラスの使用 / Use the Random class
import java.util.Scanner; // Scannerクラスの使用 / Use the Scanner class

public class MonsterBattleGame_WithMethod {

    public static void main(String args[]){
        // -----
        // プログラム実行の準備 / Program execution setup
        Scanner scan = new Scanner(System.in); // 入力処理の準備 / Prepare input handling
        Random rand = new Random(100); // 乱数生成処理の初期化
        // -----
        // Initialize random number generation
        // -----
        // パラメータ定義 / Parameters definition
        int player_hp = 100; // プレイヤーのHP / Player HP
        int monster_hp = 100; // モンスターのHP / Monster HP

        // -----
        // バトルプロセス / Battle process
        while (true) {

            System.out.println("Press Enter to attack.");
            scan.nextLine(); // Enterの入力待ち / Wait for Enter input

            // -----
            // プレイヤーの攻撃 / Attack by Player
            monster_hp = attack("Player", "Monster", monster_hp, rand);

            // 勝利判定 / Check win or lose
            if (monster_hp <= 0) {
                System.out.println("You win!");
                break;
            }

            System.out.println("---");

            // -----
            // 相手の攻撃 / Attack by Monster
            player_hp = attack("Monster", "Player", player_hp, rand);

            // 勝利判定 / Check win or lose
            if (player_hp <= 0) {
                System.out.println("You lose ...");
                break;
            }
        }
    }

    // -----
    // 攻撃処理 / Attack process
    public static int attack(String attacker_name, String target_name, int target_hp, Random rand){

        int max_random_val = 100; // クリティカル判定用乱数の上限 / Upper bound for critical hit random value
        int max_attack_point = 30; // 攻撃ダメージの上限 / Upper bound for attack damage
        int attack_multiplier = 1; // 通常時の攻撃倍率 / Normal attack multiplier
        int critical_hit_multiplier = 2; // クリティカルヒット倍率 / Critical hit multiplier
        int critical_hit_rate = 30; // クリティカルヒット発生率 / Critical hit chance

        // 攻撃者を表示 / Show attacker
        System.out.println(attacker_name + "の攻撃 / Attack by " + attacker_name);

        // クリティカルヒット判定 / Check critical hit
        if (rand.nextInt(max_random_val) < critical_hit_rate) {
            attack_multiplier = critical_hit_multiplier;
            System.out.println("> クリティカルヒット! / Critical hit!");
        }

        // 攻撃ダメージを計算 / Calculate attack damage
        int attack_damage = attack_multiplier * rand.nextInt(max_attack_point);

        // 攻撃対象のHPを減らす / Reduce target HP
        target_hp -= attack_damage;

        // 攻撃結果を表示 / Show attack result
        System.out.println("> " + target_name + "に" + attack_damage + "のダメージ! / Dealt " + attack_damage + " damage to the " + target_name + "!");
        System.out.println("> " + target_name + "の残りHP: " + target_hp + " / " + target_name + " HP: " + target_hp);

        // 更新後のHPを返す / Return updated HP
        return target_hp;
    }
}
```

次のページと比較して, By comparing with next page,

講義内課題 B In class Assignment B

Assignment8B.java

```
import java.util.Random; // Randomクラスの使用 / Use the Random class
import java.util.Scanner; // Scannerクラスの使用 / Use the Scanner class

public class Assignment8B {

    public static void main(String args[]){
        // -----
        // プログラム実行の準備 / Program execution setup
        Scanner scan = new Scanner(System.in); // 入力処理の準備 / Prepare input handling
        Random rand = new Random(999); // 乱数生成処理の初期化
        // -----
        // パラメータ定義 / Parameters definition
        int player_hp = 200; // プレイヤーのHP / Player HP
        int monster_1_hp = 100; // モンスター1のHP / Monster1 HP
        int monster_2_hp = 100; // モンスター2のHP / Monster2 HP
        // -----
        // バトルプロセス / Battle process
        while (true) {

            System.out.println("Press Enter to attack.");
            scan.nextLine(); // Enterの入力待ち / Wait for Enter input

            // -----
            // プレイヤーの攻撃 / Attack by Player
            if (monster_1_hp > 0) { monster_1_hp = attack("Player", "Monster1", monster_1_hp, rand); }
            if (monster_2_hp > 0) { monster_2_hp = attack("Player", "Monster2", monster_2_hp, rand); }

            // 勝敗判定 / Check win or lose
            if (monster_1_hp <= 0 && monster_2_hp <= 0) {
                System.out.println("You win!");
                break;
            }

            System.out.println("---");

            // -----
            // 相手の攻撃 / Attack by Monster
            if (monster_1_hp > 0) { player_hp = attack("Monster1", "Player", player_hp, rand); }
            if (monster_2_hp > 0) { player_hp = attack("Monster2", "Player", player_hp, rand); }

            // 勝敗判定 / Check win or lose
            if (player_hp <= 0) {
                System.out.println("You lose ...");
                break;
            }
        }
    }

    // =====
    // 攻撃処理 / Attack process
    public static int attack(String attacker_name, String target_name, int target_hp, Random rand){

        int max_random_val = 100; // 確率判定用乱数の上限 / Upper bound for probability check
        int max_attack_point = 30; // 攻撃ダメージの上限 / Upper bound for attack damage
        int attack_multiplier = 1; // 通常時の攻撃倍率 / Normal attack multiplier
        int critical_hit_multiplier = 2; // クリティカルヒット倍率 / Critical hit multiplier
        int critical_hit_rate = 30; // クリティカルヒット発生率 / Critical hit chance
        int attack_fails_rate = 20; // 攻撃失敗確率 / Attack failure probability
        int attack_fails_multiplier = 0; // 攻撃失敗時の攻撃倍率 / Multiplier on the attack failure

        // 攻撃者を表示 / Show attacker
        System.out.println(attacker_name + "の攻撃 / Attack by " + attacker_name);

        // 攻撃失敗判定 / Check attack failure
        if (rand.nextInt(max_random_val) < attack_fails_rate) {
            attack_multiplier = attack_fails_multiplier;
            System.out.println("> 攻撃失敗...! / Attack failed...!");
        }

        // クリティカルヒット判定 / Check critical hit
        if (attack_multiplier != 0 && rand.nextInt(max_random_val) < critical_hit_rate) {
            attack_multiplier = critical_hit_multiplier;
            System.out.println("> クリティカルヒット! / Critical hit!");
        }

        // 攻撃ダメージを計算 / Calculate attack damage
        int attack_damage = attack_multiplier * rand.nextInt(max_attack_point);

        // 攻撃対象のHPを減らす / Reduce target HP
        target_hp -= attack_damage;

        // 攻撃結果を表示 / Show attack result
        System.out.println("> " + target_name + "に" + attack_damage + "のダメージ! / Dealt " + attack_damage + " damage to the " + target_name + "!");
        System.out.println("> " + target_name + "の残りHP: " + target_hp + " / " + target_name + " HP: " + target_hp);

        // 更新後のHPを返す / Return updated HP
        return target_hp;
    }
}
```

差分を確認して下さい Please check the differences.

講義内課題 C In class Assignment C

メソッド化の目的はコードの共通化だけではありません。

メソッド化によって得られるメリットをパワーポイント形式(**スライド5枚以内**)で説明しなさい。

その際、必ず文献から引用する形で回答を作成すること。

また、文献は IEEE Reference Guide に示されたフォーマットを用いて、明示すること。

<https://journals.ieeeauthorcenter.ieee.org/your-role-in-article-production/ieee-editorial-style-manual/>

The purpose of using methods is not limited to code reuse.

Explain the benefits of using methods in PowerPoint format (**within 5 slides**).

In your explanation, you must cite relevant literature.

The references must be clearly listed using the format specified in the IEEE Reference Guide.

<https://journals.ieeeauthorcenter.ieee.org/your-role-in-article-production/ieee-editorial-style-manual/>

提出ファイル名 Submission file name: **Assignment8C_[Student ID].pptx or other powerpoint filetype**

提出方法 Submission method: **Moodle**

提出期限 Submission deadline: **第9回講義の開始まで. By the start of the class 9.**



クラス・インスタンス

Class, Instance

クラス Class

- データと処理をまとめ、インスタンスを生成するための雛形
A template for grouping data and operations and creating instances.

クラス Class



フィールド Field

名前：早稲田太郎	Name: Taro Waseda
誕生年：1999	BirthYear: 1999
誕生月：5	BirthMonth: 5
誕生日：20	BirthDay: 20
身長：165.8 cm	Height: 165.8 cm

メソッド Method

- Walk()
- Run()
- talk()
- ⋮

関連のあるデータを
まとめて扱いたい！
Related data should be
handled together.

クラス Class

- データと処理をまとめ、インスタンスを生成するための雛形
A template for grouping data and operations and creating instances.

クラス Class



フィールド Field

名前：早稲田太郎 Name: Taro Waseda

誕生年：1999 BirthYear: 1999

誕生月：5 BirthMonth: 5

誕生日：20 BirthDay: 20

身長：165.8 cm Height: 165.8 cm

メソッド Method

- Walk()
- Run()
- talk()
- ⋮

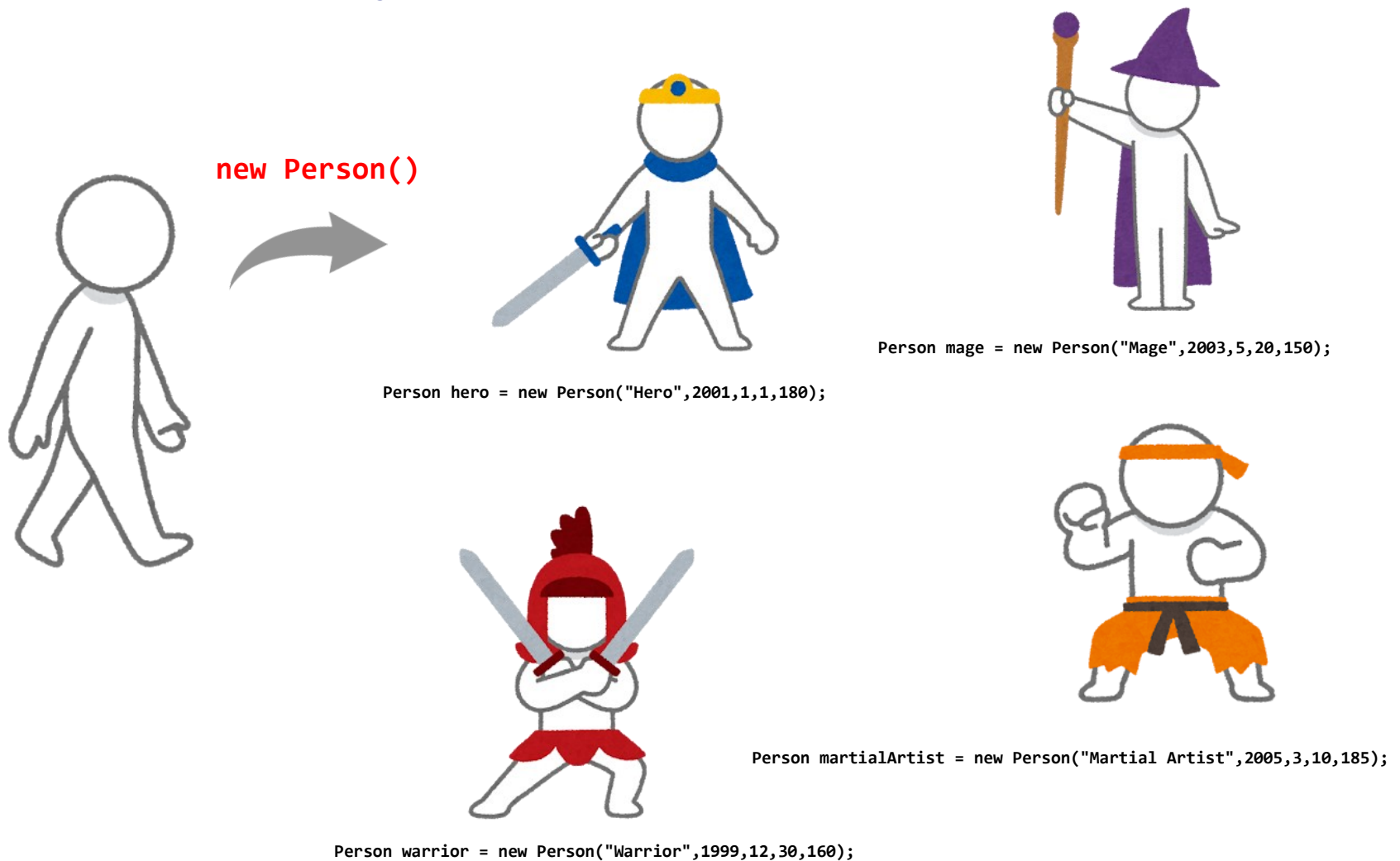
関連のあるデータを
まとめて扱いたい！
Related data should be
handled together.

Class (クラス) : public class Person

```
public class Person {  
  
    // -----  
    // フィールド / Fields  
    // -----  
    String name;  
    int birth_year;  
    int birth_month;  
    int birth_day;  
    double height;  
  
    // -----  
    // コンストラクタ / Constructor (初期化 / Initialization)  
    // -----  
    public Person(String n, int y, int m, int d, double h) {  
        name = n;  
        birth_year = y;  
        birth_month = m;  
        birth_day = d;  
        height = h;  
    }  
  
    // -----  
    // メソッド / Method  
    // -----  
    public void walk() {  
        /* Walking process */  
    }  
  
    public void run() {  
        /* Running process */  
    }  
  
    public void talk() {  
        /* Talking process */  
    }  
}
```

インスタンス Instance

- インスタンスとは, クラスの実体
An instance is an object created from a class.



クラスの練習 Practice Using Classes

右の3ファイルを同じディレクトリ内に作成して下さい。
Please create the following three files in the same directory.

1. MonsterBattleGame_WithClass.java
2. GameCharacter.java
3. BattleProcess.java

MonsterBattleGame_WithClass.java

```
import java.util.Random; // Randomクラスの使用 / Use the Random class
import java.util.Scanner; // Scannerクラスの使用 / Use the Scanner class

public class MonsterBattleGame_WithClass {

    public static void main(String args[]){ // プログラムの実行開始地点 / The starting point of program execution
        // プログラム実行の準備 / Program execution setup
        Scanner scan = new Scanner(System.in); // Enter入力を受け取るための準備 / Prepare for reading Enter input
        Random rand = new Random(100); // 乱数生成処理の初期化 / Initialize random number generation

        // -----
        // インスタンスの生成と参照 / Creating and referring to instances
        GameCharacter player = new GameCharacter("Player", 100, 30); // GameCharacterクラスの実体(インスタンス)を生成し、変数playerから参照 / Create an instance of the GameCharacter class and refer to it from the variable player
        GameCharacter monster = new GameCharacter("Monster", 100, 30); // GameCharacterクラスの実体(インスタンス)を生成し、変数monsterから参照 / Create an instance of the GameCharacter class and refer to it from the variable monster

        // ゲームタイトルの表示 / Display the game title
        System.out.println("~~~~~");
        System.out.println("      MonsterBattleGame_WithClass      ");
        System.out.println("~~~~~");

        player.print(); // プレイヤー情報の表示 / Display the player info
        monster.print(); // モンスター情報の表示 / Display the monster info

        // -----
        // バトル進行 / Battle control
        System.out.println("# バトル開始 / Battle start");
        while (true) {

            System.out.println("# Enterを入力して攻撃 / Press Enter to attack");
            scan.nextLine(); // プレイヤーの入力待ち / Wait for player input

            // -----
            // プレイヤーがモンスターを攻撃 / The player attacks the monster
            BattleProcess.attack(player, monster, rand);

            // 戦闘不能判定 / Check whether the character is defeated
            if (BattleProcess.isDefeated(monster)) { // モンスターが戦闘不能の場合 / If monster is defeated
                BattleProcess.printResult(player, monster); // プレイヤーの勝利 / Player win
                break; // ループを抜ける / Exit the loop
            }

            // -----
            // モンスターがプレイヤーを攻撃 / The monster attacks the player
            BattleProcess.attack(monster, player, rand);

            // 戦闘不能判定 / Check whether the character is defeated
            if (BattleProcess.isDefeated(player)) { // プレイヤーが戦闘不能の場合 / If player is defeated
                BattleProcess.printResult(monster, player); // モンスターの勝利 / Monster win
                break; // ループを抜ける / Exit the loop
            }

        }

        // -----
        // 終了処理 / Exit handling
        scan.close(); // Scannerを閉じる / Close the Scanner
    }
}
```

クラスの練習 Practice Using Classes

右の3ファイルを同じディレクトリ内に作成して下さい。
Please create the following three files in the same directory.

1. MonsterBattleGame_WithClass.java
2. GameCharacter.java
3. BattleProcess.java

GameCharacter.java

```
// =====  
//  
// ゲームキャラクタークラス / Game character class  
//  
// =====  
public class GameCharacter {  
  
    // -----  
    // フィールド / Fields  
    // - インスタンスが持つデータ / Fields are data stored in each instance  
    String name;      // キャラクター名 / Character name  
    int hp;           // キャラクターのHP / Character HP  
    int attack_power; // キャラクターの攻撃力 / Character Attack power  
  
    // -----  
    // コンストラクタ / Constructor  
    // - コンストラクタの名前はクラス名と同じにする / The constructor name must be the same as the class name  
    // - コンストラクタはインスタンスを生成するときに最初に実行される / A constructor runs when an instance is created  
    public GameCharacter(String character_name, int character_hp, int character_attack_power) {  
        name = character_name;      // 引数character_nameをフィールドnameに代入 / Assign the argument character_name to the field name  
        hp = character_hp;          // 引数character_hpをフィールドhpに代入 / Assign the argument character_hp to the field hp  
        attack_power = character_attack_power; // 引数character_attack_powerをフィールドattack_powerに代入 / Assign the argument character_attack_power to the field attack_power  
    }  
  
    // -----  
    // パラメータを表示するメソッド / Method for printing parameters  
    public void print() {  
        System.out.println("[ " + name + " ]");  
        System.out.println("- HP: " + hp);  
        System.out.println("- Attack Power: " + attack_power);  
        System.out.println("-----");  
    }  
  
    // -----  
    // ダメージを受けるメソッド / Method for receiving damage  
    public void receiveDamage(int damage) {  
        hp -= damage; // 受けたダメージ分だけHPを減らす / Reduce HP by the damage value  
    }  
}
```

クラスの練習 Practice Using Classes

右の3ファイルを同じディレクトリ内に作成して下さい。
Please create the following three files in the same directory.

1. MonsterBattleGame_WithClass.java
2. GameCharacter.java
3. BattleProcess.java

BattleProcess.java

```
import java.util.Random; // Randomクラスの使用 / Use the Random class

// =====
//
// 戦闘プロセスクラス / Battle process class
//
// - BattleProcessはHPなどのフィールドを持たないため、インスタンスを作らずに使う / Since BattleProcess has no fields such as HP, these methods are used without creating an instance
// - staticを付けると、BattleProcessインスタンスを作らずにメソッドを呼び出せる / By using static, this method can be called without creating a BattleProcess instance
//
// =====
public class BattleProcess {

    // -----
    // 攻撃処理メソッド / Attack process method
    // - attackerがtargetを攻撃 / The attacker attacks the target
    public static void attack(GameCharacter attacker, GameCharacter target, Random rand) {

        // -----
        // 攻撃者を表示 / Show the attacker
        System.out.println(attacker.name + "の攻撃 / Attack by " + attacker.name);

        // -----
        // 攻撃ダメージの計算 / Calculate attack damage
        int attack_damage = rand.nextInt(attacker.attack_power);

        // -----
        // 攻撃対象にダメージを与える / Deal damage to the target
        target.receiveDamage(attack_damage);

        // -----
        // 攻撃結果を表示 / Show attack result
        System.out.println("> " + target.name + "に" + attack_damage + "のダメージ! / Dealt " + attack_damage + " damage to the " + target.name + "!");
        System.out.println("> " + target.name + "の残りHP: " + target.hp + " / " + target.name + " HP: " + target.hp);
    }

    // 戦闘不能を判定するメソッド / Method for checking whether the character is defeated
    public static boolean isDefeated(GameCharacter character) {
        return character.hp <= 0; // HPが0以下ならtrueを返す / Return true if HP is 0 or less
    }

    // 戦闘結果を表示するメソッド / Method for displaying the battle result
    public static void printResult(GameCharacter winner, GameCharacter loser) {
        System.out.println("");
        System.out.println("-----");
        System.out.println("結果 / Battle Result");
        System.out.println("-----");
        System.out.println("> 勝利 / Win : " + winner.name);
        System.out.println("> 敗北 / Lose: " + loser.name);
    }
}
```

クラスの練習 Practice Using Classes

右の3ファイルを同じディレクトリ内に作成して下さい。
Please create the following three files in the same directory.

1. MonsterBattleGame_WithClass.java
2. GameCharacter.java
3. BattleProcess.java

コンパイルして実行します。
Compile and execute.

Terminal

```
$ javac MonsterBattleGame_WithClass.java GameCharacter.java BattleProcess.java ← まとめてコンパイル / Compile them together
$ java MonsterBattleGame_WithClass ← 実行 / Execution
```

```
● 07173004.2005:~/java/class05$ javac MonsterBattleGame_WithClass.java GameCharacter.java BattleProcess.java
● 07173004.2005:~/java/class05$ java MonsterBattleGame_WithClass

-----
MonsterBattleGame_WithClass
-----

[ Player ]
- HP: 100
- Attack Power: 30
-----
[ Monster ]
- HP: 100
- Attack Power: 30
-----

# バトル開始 / Battle start
# Enterを入力して攻撃 / Press Enter to attack

Playerの攻撃 / Attack by Player
> Monsterに25のダメージ! / Dealt 25 damage to the Monster!
> Monsterの残りHP: 75 / Monster HP: 75
Monsterの攻撃 / Attack by Monster
> Playerに10のダメージ! / Dealt 10 damage to the Player!
> Playerの残りHP: 90 / Player HP: 90
# Enterを入力して攻撃 / Press Enter to attack

Playerの攻撃 / Attack by Player
> Monsterに40のダメージ! / Dealt 4 damage to the Monster!
> Monsterの残りHP: 71 / Monster HP: 71
Monsterの攻撃 / Attack by Monster
> Playerに10のダメージ! / Dealt 18 damage to the Player!
> Playerの残りHP: 72 / Player HP: 72
# Enterを入力して攻撃 / Press Enter to attack

Playerの攻撃 / Attack by Player
> Monsterに10のダメージ! / Dealt 1 damage to the Monster!
> Monsterの残りHP: 70 / Monster HP: 70
Monsterの攻撃 / Attack by Monster
> Playerに6のダメージ! / Dealt 6 damage to the Player!
> Playerの残りHP: 66 / Player HP: 66
# Enterを入力して攻撃 / Press Enter to attack

Playerの攻撃 / Attack by Player
> Monsterに26のダメージ! / Dealt 26 damage to the Monster!
> Monsterの残りHP: 44 / Monster HP: 44
Monsterの攻撃 / Attack by Monster
> Playerに8のダメージ! / Dealt 8 damage to the Player!
> Playerの残りHP: 58 / Player HP: 58
# Enterを入力して攻撃 / Press Enter to attack

Playerの攻撃 / Attack by Player
> Monsterに13のダメージ! / Dealt 13 damage to the Monster!
> Monsterの残りHP: 31 / Monster HP: 31
Monsterの攻撃 / Attack by Monster
> Playerに13のダメージ! / Dealt 11 damage to the Player!
> Playerの残りHP: 45 / Player HP: 45
# Enterを入力して攻撃 / Press Enter to attack

Playerの攻撃 / Attack by Player
> Monsterに22のダメージ! / Dealt 22 damage to the Monster!
> Monsterの残りHP: 9 / Monster HP: 9
Monsterの攻撃 / Attack by Monster
> Playerに17のダメージ! / Dealt 17 damage to the Player!
> Playerの残りHP: 28 / Player HP: 28
# Enterを入力して攻撃 / Press Enter to attack

Playerの攻撃 / Attack by Player
> Monsterに26のダメージ! / Dealt 26 damage to the Monster!
> Monsterの残りHP: -17 / Monster HP: -17

-----
結果 / Battle Result
-----
> 勝利 / win : Player
> 敗北 / lose: Monster
● 07173004.2005:~/java/class05$
```

課題が終われば退出可能です。
TAさんに自身の座席番号を伝えてからご退出下さい。

You can leave once you have completed the assignment. Please inform
a teaching assistant (TA) of your seat number before leaving.

講義内課題

In class Assignment

Please do not include your CD (check digit)
in the student ID part of the filename.

Example:

Filename Format: Assignment9A_[Student ID].java

Student ID: 1w123456-7

Filename: Assignment9A_1w123456.java

講義内課題 A In class Assignment A

- 右の3ファイルを作成して下さい。
Please create the following three files.
 - 1. MonsterBattleGame_WithClass.java
 - 2. GameCharacter.java
 - 3. BattleProcess.java
- HPが0未満にならないように、メソッドreceiveDamageを修正して下さい。
Modify the receiveDamage method to ensure that HP never becomes less than 0.

下で指定されたコマンドでコンパイル&実行できるように、ファイルを修正して下さい。
コンパイル&実行できる状態で提出して下さい。

Modify the file so that it can be compiled and run using the following commands.
Please submit the program that can be compiled and run.

Commands

```
$ javac Assignment9A_[Student ID].java GameCharacter.java BattleProcess.java  
$ java Assignment9A_[Student ID]
```

提出ファイル名 Submission file name: Assignment9A_[Student ID].java, GameCharacter.java BattleProcess.java

提出方法 Submission method: Moodle

提出期限 Submission deadline: 第10回講義の開始まで. By the start of the class 10.

講義内課題 B In class Assignment B

3体の敵モンスターと戦うゲーム, `MonsterBattleGame_WithClass_2.java` を作成しなさい。

乱数のシードは999, Classを必ず使用してかつ以下の条件を満たすこと。

- プレイヤーのHPは300, 攻撃力は60. モンスターのHPは100, 攻撃力は20.
- プレイヤーは20%, モンスターは10%の確率で攻撃が失敗するようにプログラムすること.
- 攻撃順: プレイヤーが各モンスターを攻撃 → 各モンスターがプレイヤーを攻撃 → ...

提出の際は, 下で指定されたコマンドでコンパイル&実行できる状態で提出して下さい。

Create `MonsterBattleGame_WithClass_2.java`, a game in which the player fights against three enemy monsters.

Set the random seed to 999, and the program must be implemented using class and the following conditions must be satisfied.

- The player has 300 HP and an attack power of 60. The monster has 100 HP and an attack power of 20.
- Implement attack failure so that the player's attack fails with a 20% probability and the monster's attack fails with a 10% probability.
- Attack order : The player attacks each monster → Each monster attacks the player → ...

When submitting, make sure that your files can be compiled and run using following commands.

Commands

```
$ javac Assignment9B_[Student ID].java GameCharacter.java BattleProcess.java
$ java Assignment9B_[Student ID]
```

提出ファイル名 Submission file name: `Assignment9B_[Student ID].java, GameCharacter.java BattleProcess.java`

提出方法 Submission method: `Moodle`

提出期限 Submission deadline: `第10回講義の開始まで. By the start of the class 10.`

講義内課題 C In class Assignment C

講義内課題Bで工夫した点をパワーポイント形式(スライド5枚以内)で説明しなさい。

Explain the improvements you made in In class Assignment B in PowerPoint format (within 5 slides).

提出ファイル名 Submission file name: Assignment9C_[Student ID].pptx or other PowerPoint filetype

提出方法 Submission method: Moodle

提出期限 Submission deadline: 第10回講義の開始まで. By the start of the class 10.